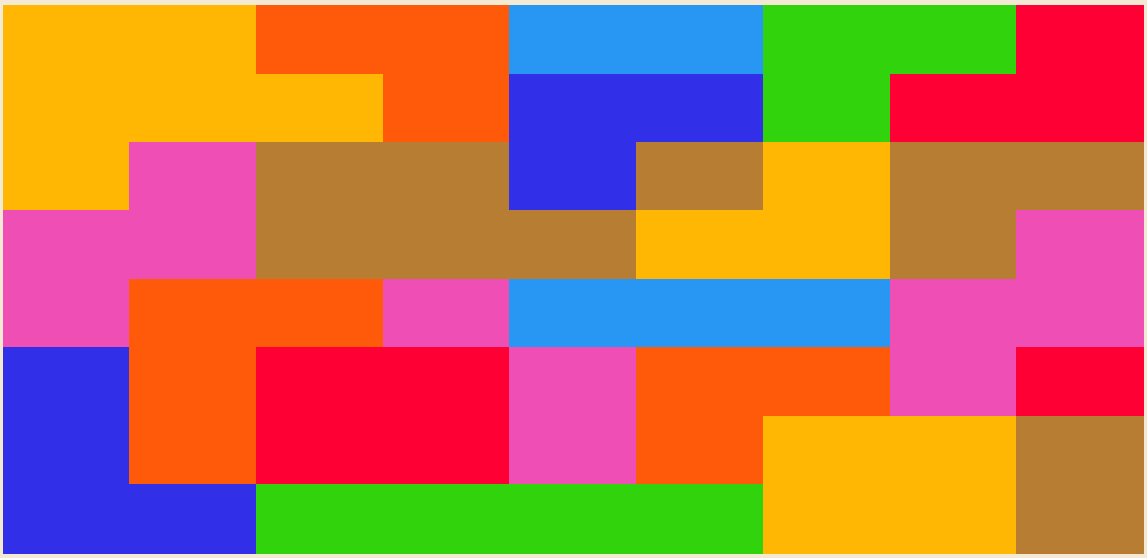

EXOPLANET
MISSION CONTROL

Build it. Understand it. Explore the data.

PYTHON + PANDAS + STREAMLIT

Build a browser-based scientific data explorer with Python, pandas, and Streamlit.

Your app will read 720 synthetic exoplanet observations from a CSV file. A user will choose two measurements, and the app will rebuild an interactive point chart around those choices. By the end, you will have written a small data pipeline: read a file, prepare a table, accept user input, clean selected values, and present the result in a browser.

The program you are building

Your completed page will:

- locate the CSV reliably
- read the survey into a pandas DataFrame
- display the survey record count
- offer X-axis and Y-axis menus
- begin with planet radius and planet mass selected
- convert selected measurements into numeric values
- remove rows that cannot appear on the chosen axes
- color observations by planet class
- display the first ten source rows beneath the chart

The program's data path

REFERENCE

```
CSV file
   |
   v
load_data()
   |
   v
pandas DataFrame
   |
   v
sidebar choices
   |
   v
cleaned chart_data
   |
   v
interactive chart + source table
```

Each function owns one part of that path. `load_data()` prepares the full survey. `show_chart(df)` prepares and displays the selected comparison. `setup()` arranges the browser page and calls the other functions in the correct order.

Project folder

Keep these files together:

REFERENCE

```
S9LFINAL/
|-- main.py
|-- README.md
|-- requirements.txt
`-- synthetic_exoplanet_survey.csv
```

The CSV must remain beside `main.py`. The code will locate it from the script's own folder rather than relying on the terminal's current folder.

Windows setup

Open PowerShell inside `S9LFINAL`.

1. Create a virtual environment

POWERSHELL

```
py -m venv .venv
```

A virtual environment is a project-specific Python installation. Packages installed inside `.venv` belong to this project, so another Python project can use different versions without interfering. You create the environment once, then activate it whenever you return to the project.

2. Activate the environment

POWERSHELL

```
.\.venv\Scripts\Activate.ps1
```

Your PowerShell prompt should begin with:

REFERENCE

```
(.venv)
```

That prefix confirms that `python` and `pip` now refer to the project environment.

When PowerShell blocks activation, run:

POWERSHELL

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

`-Scope Process` limits the policy change to the current PowerShell window.

3. Prepare `requirements.txt`

Type:

REFERENCE

```
streamlit
pandas
```

`requirements.txt` is the project's dependency list. It lets another computer install the same libraries without guessing what the program needs.

4. Install the packages

POWERSHELL

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

5. Start the app

POWERSHELL

```
python -m streamlit run main.py
```

Streamlit runs a local web server and opens the page in a browser. Keep PowerShell open while the app is running. Running `python main.py` directly skips the Streamlit session and can produce bare-mode warnings.

Checkpoint 1: imports and a reliable CSV path

FILE LEVEL, ABOVE EVERY FUNCTION.

```
from pathlib import Path

import pandas as pd
import streamlit as st

DATA_FILE = Path(__file__).with_name(
    "synthetic_exoplanet_survey.csv"
)
```

What is happening

Imports make code from other modules available in your file. `Path` handles file locations, pandas handles tables, and Streamlit builds the browser interface.

`__file__` represents the location of `main.py`. `Path(__file__).with_name(...)` keeps that folder and replaces the filename with the CSV filename. This means the app can locate the survey even when it is launched from a different folder.

`DATA_FILE` is written in uppercase because it is a constant: a value chosen once and reused without being changed.

Checkpoint 2: load the survey

CREATE A NEW FUNCTION NAMED `LOAD_DATA()`.

```
@st.cache_data
def load_data():
    df = pd.read_csv(DATA_FILE)
    df.columns = df.columns.str.strip().str.lower()
    return df
```

What is happening

`pd.read_csv(DATA_FILE)` turns the CSV into a DataFrame named `df`. A DataFrame is a table whose rows represent observations and whose columns represent properties.

The header-cleanup statement applies two string operations to every column name. `strip()` removes accidental spaces at the beginning or end, and `lower()` changes letters to lowercase. The rest of your code can now use predictable names such as `planet_mass_earth`.

`return df` sends the prepared table back to whichever part of the program calls the function.

Why cache the function

Streamlit reruns the script whenever a menu changes. `@st.cache_data` allows Streamlit to reuse the loaded table when the file and function inputs have not changed. The menus still update, but the CSV does not need to be read from storage repeatedly.

Checkpoint 3: begin `show_chart(df)`

CREATE A NEW FUNCTION NAMED `SHOW_CHART(DF)`. EVERYTHING INDENTED BENEATH IT BELONGS TO THAT FUNCTION.

```
def show_chart(df):
    chart_columns = [
        "distance_light_years",
        "star_mass_solar",
        "star_radius_solar",
        "star_temperature_k",
        "planet_radius_earth",
        "planet_mass_earth",
        "orbital_period_days",
        "semi_major_axis_au",
        "eccentricity",
        "equilibrium_temperature_k",
        "transit_depth_ppm",
        "signal_strength",
        "detection_confidence_pct",
        "liquid_water_index",
        "moon_candidate_count"
    ]
```

What is happening

The parameter `df` gives the function access to the loaded survey. The function does not read the CSV again; it receives the existing DataFrame from `setup()`.

`chart_columns` is a curated list of measurements that make sense on numeric axes. The survey also contains names, notes, categories, and dates. Including every column would create confusing menus and invalid chart choices. This list defines the part of the dataset your interface is designed to explore.

Python preserves the order of the list, so the sidebar will present the measurements in this same order.

Checkpoint 4: keep valid columns and add the axis menus

CONTINUE INSIDE `SHOW_CHART(DF)`. KEEP FOUR SPACES BEFORE EVERY LINE. DO NOT WRITE A SECOND `DEF SHOW_CHART(DF):` LINE.

```
    # ... continue inside show_chart(df) ...

    chart_columns = [
        column for column in chart_columns
        if column in df.columns
    ]

    x_column = st.sidebar.selectbox(
        "Choose the X axis",
        chart_columns,
        index=chart_columns.index("planet_radius_earth")
    )

    y_column = st.sidebar.selectbox(
        "Choose the Y axis",
        chart_columns,
        index=chart_columns.index("planet_mass_earth")
    )
```

What is happening

The list comprehension compares the curated list with the real DataFrame. A name survives only when it appears in `df.columns`. This protects the menus from a missing or renamed column.

Each `selectbox` displays the same options and returns the user's current choice. The returned strings are stored in `x_column` and `y_column`. Those variables might contain `"planet_radius_earth"` now and a different column name after the user changes a menu.

The `index` arguments select a meaningful starting pair, so the page opens with a chart instead of an unfinished state.

Streamlit's rerun model

A menu choice causes Streamlit to rerun `main.py` from top to bottom. During the new run, the select boxes return the latest choices. The code below them rebuilds `chart_data` and redraws the chart. The app feels interactive even though Python is executing the script again.

Checkpoint 5: create a smaller chart table

REMAIN INSIDE `SHOW_CHART(DF)`.

```
    # ... still inside show_chart(df) ...

    chart_data = df[
        [x_column, y_column, "planet_class"]
    ].copy()
```

What is happening

The full survey has many columns, while the chart needs only three: the selected X measurement, the selected Y measurement, and the category used for color.

The expression inside `df[...]` selects those three columns and produces a smaller DataFrame. `.copy()` creates an independent chart table. The next statements can convert values inside `chart_data` without modifying the original survey displayed later on the page.

This is a common data-analysis pattern: keep the full source table intact and build a smaller working table for a specific task.

Checkpoint 6: convert and clean the selected measurements

REMAIN INSIDE `SHOW_CHART(DF)`.

```
    chart_data[x_column] = pd.to_numeric(
        chart_data[x_column],
        errors="coerce"
    )

    chart_data[y_column] = pd.to_numeric(
        chart_data[y_column],
        errors="coerce"
    )

    chart_data = chart_data.dropna(
        subset=[x_column, y_column]
    )
```

What is happening

CSV values may look numeric while still being stored as text. `pd.to_numeric(...)` asks pandas to convert the selected columns into real numeric values.

`errors="coerce"` gives pandas a safe rule for unexpected text: replace it with a missing value rather than stopping the entire app. `dropna(...)` then removes only the rows missing one of the selected coordinates.

The cleanup is dynamic. Changing either menu changes which columns are converted and which rows are usable. The original `df` remains unchanged because the conversion happens inside `chart_data`.

Checkpoint 7: draw the dynamic chart

THESE ARE THE FINAL INDENTED LINES IN `SHOW_CHART(DF)`.

```
    # ... final lines inside show_chart(df) ...

    st.subheader(
        f"{y_column} compared with {x_column}"
    )

    st.scatter_chart(
        chart_data,
        x=x_column,
        y=y_column,
        color="planet_class",
        height=500
    )
```

What is happening

The f-string inserts the current variable values into the heading. The title therefore describes the selected comparison instead of remaining fixed.

`st.scatter_chart(...)` receives the cleaned DataFrame and the names of the columns to use. Each visible point represents one row with usable X and Y values. Horizontal position comes from `x_column`, vertical position comes from `y_column`, and color comes from `planet_class`.

The function ends when indentation returns to the left edge.

Checkpoint 8: build the page with `setup()`

RETURN TO THE LEFT EDGE AND START A NEW FUNCTION.

```
def setup():
    st.set_page_config(
        page_title="Exoplanet Mission Control",
        layout="wide"
    )

    st.title("Exoplanet Mission Control")
    st.caption(
        "Choose two measurements and explore their relationship."
    )

    df = load_data()

    st.write("Total records:", len(df))

    show_chart(df)

    st.subheader("Survey Data")
    st.dataframe(
        df.head(10),
        width="stretch"
    )
```

What is happening

`setup()` controls the page's reading order. Configuration must happen before other Streamlit page commands. The title and caption establish the app's purpose.

`df = load_data()` calls your data-loading function and stores its returned DataFrame. `len(df)` counts rows. `show_chart(df)` passes the table into the visualization function.

`df.head(10)` returns the first ten rows for inspection. The chart summarizes patterns, while the table exposes the source values behind those patterns.

Checkpoint 9: start the program

FILE LEVEL, OUTSIDE EVERY FUNCTION.

```
if __name__ == "__main__":
    setup()
```

What is happening

Python gives a directly executed file the special name `"__main__"`. This condition calls `setup()` when Streamlit runs `main.py`.

Keeping startup code at the bottom makes the file easier to read: first imports and constants, then function definitions, then the single instruction that begins the program.

What you should see

- Total records: 720
- many scientific options in both sidebar menus
- `planet_radius_earth` selected for X
- `planet_mass_earth` selected for Y

- a colored point chart
- ten survey rows beneath the chart

Change either menu. Streamlit should rerun the script, update the heading, prepare a new chart table, and redraw the comparison.

Read a chart like a scientist

Begin by naming both axes. Then look for the overall direction, clusters, empty regions, and observations far from the main pattern.

Try these comparisons:

X axis	Y axis	Question
`planet_radius_earth`	`planet_mass_earth`	Do larger worlds usually have greater mass?
`semi_major_axis_au`	`orbital_period_days`	How does orbital distance relate to orbital time?
`star_temperature_k`	`equilibrium_temperature_k`	Do hotter stars tend to have hotter planets?
`distance_light_years`	`detection_confidence_pct`	Does distance appear related to detection confidence?
`equilibrium_temperature_k`	`liquid_water_index`	Where do higher water-index values appear?

Use careful language:

> **As _____ changes, _____ tends to _____. Some observations do not follow that pattern, so I would examine _____ next.**

A chart can reveal association, but it does not prove that one measurement caused the other.

Complete `main.py`

PYTHON - PART 1 OF 4

```python
from pathlib import Path

import pandas as pd
import streamlit as st

DATA_FILE = Path(__file__).with_name(
    "synthetic_exoplanet_survey.csv"
)

@st.cache_data
def load_data():
    df = pd.read_csv(DATA_FILE)
    df.columns = df.columns.str.strip().str.lower()
    return df

def show_chart(df):
    chart_columns = [
        "distance_light_years",
        "star_mass_solar",
        "star_radius_solar",
        "star_temperature_k",
        "planet_radius_earth",
        "planet_mass_earth",
        "orbital_period_days",
        "semi_major_axis_au",
        "eccentricity",
        "equilibrium_temperature_k",
        "transit_depth_ppm",
```

PYTHON - PART 2 OF 4

```python
        "signal_strength",
        "detection_confidence_pct",
        "liquid_water_index",
        "moon_candidate_count"
    ]

    chart_columns = [
        column for column in chart_columns
        if column in df.columns
    ]

    x_column = st.sidebar.selectbox(
        "Choose the X axis",
        chart_columns,
        index=chart_columns.index("planet_radius_earth")
    )

    y_column = st.sidebar.selectbox(
        "Choose the Y axis",
        chart_columns,
        index=chart_columns.index("planet_mass_earth")
    )

    chart_data = df[
        [x_column, y_column, "planet_class"]
    ].copy()

    chart_data[x_column] = pd.to_numeric(
```

PYTHON - PART 3 OF 4

```
        chart_data[x_column],
        errors="coerce"
    )

    chart_data[y_column] = pd.to_numeric(
        chart_data[y_column],
        errors="coerce"
    )

    chart_data = chart_data.dropna(
        subset=[x_column, y_column]
    )

    st.subheader(
        f"{y_column} compared with {x_column}"
    )

    st.scatter_chart(
        chart_data,
        x=x_column,
        y=y_column,
        color="planet_class",
        height=500
    )

def setup():
    st.set_page_config(
        page_title="Exoplanet Mission Control",
```

PYTHON - PART 4 OF 4

```
        layout="wide"
    )

    st.title("Exoplanet Mission Control")
    st.caption(
        "Choose two measurements and explore their relationship."
    )

    df = load_data()

    st.write("Total records:", len(df))

    show_chart(df)

    st.subheader("Survey Data")
    st.dataframe(
        df.head(10),
        width="stretch"
    )

if __name__ == "__main__":
    setup()
```

Recap: how the program works

1. `Path` builds a reliable location for the CSV.
2. `load_data()` reads the file and standardizes its column names.
3. `setup()` receives the complete DataFrame and arranges the page.
4. `show_chart(df)` receives the same DataFrame.
5. The sidebar stores two column names in `x_column` and `y_column`.
6. A smaller DataFrame keeps only the selected measurements and planet class.
7. pandas converts the selected measurements and removes unusable rows.
8. Streamlit draws the chart and reruns the script after each menu change.

The important pattern is reusable far beyond exoplanets:

REFERENCE

```
load -> select -> clean -> visualize -> inspect
```

Replace the CSV and the curated column list, and the same structure can become a weather explorer, sports explorer, environmental dashboard, or laboratory-data viewer.

What each function owns

- `load_data()` owns file reading and column-name preparation.
- `show_chart(df)` owns axis choices, numeric conversion, missing-value cleanup, and visualization.
- `setup()` owns page order and decides when the other functions run.

This separation matters because each function has one clear responsibility. When a failure appears, you can inspect the function responsible for that stage instead of searching the entire file without a plan.

What happens after a menu change

Streamlit reruns `main.py` from the top. The imports and function definitions are read again, `setup()` runs, and `load_data()` returns its cached DataFrame. The select boxes return the newest column names. `show_chart(df)` then creates a new `chart_data` table from those choices, converts the selected values, removes unusable coordinates, and sends the new result to the chart.

How to extend the app safely

Add one improvement at a time and keep it inside the function that owns that behavior. A new measurement belongs in `chart_columns`. A new filter belongs inside `show_chart(df)` before the chart command. A new heading or metric belongs in `setup()`. Save and test after every small change so the most recent edit is easy to inspect.

Troubleshooting

`No module named streamlit`

POWERSHELL

```
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
```

PowerShell refuses to activate `.venv`

POWERSHELL

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.\.venv\Scripts\Activate.ps1
```

Bare-mode or `NoSessionContext` warnings

Start the file through Streamlit:

POWERSHELL

```
python -m streamlit run main.py
```

The CSV cannot be found

Confirm that the CSV is beside `main.py` and that your code uses:

PYTHON

```
DATA_FILE = Path(__file__).with_name(
    "synthetic_exoplanet_survey.csv"
)
```

`ValueError: list.index(x): x not in list`

Confirm that the CSV contains `planet_radius_earth` and `planet_mass_earth`, and confirm that both names appear in `chart_columns`.

The page opens without a chart

Check these items:

1. `show_chart(df)` is called inside `setup()`.
2. `st.scatter_chart(...)` remains indented inside `show_chart(df)`.
3. `x=x_column` and `y=y_column` are present.
4. The app was started with `python -m streamlit run main.py`.
5. The first terminal error has been read from top to bottom.